

CSCI 423

Introduction to Algorithms



DR. RAAFAT ELFOULY



Chapter 4

SORTING ALGORITHMS

Chapter Outline

3

- Bubble sort
- Insertion sort
- Merge sort
- Quicksort

Optional HW 4 Due October 30th

(Team HW) (extra credit)



- Add binary search to insertion sort to reduce overall execution time.
- Update the algorithm and write the code for it.
- Test the code for list sizes 10K, 100K and 1 M element.
- Draw the execution time curve for the original insertion sort and the modified one (with binary search) as a function of input size(s)
- Deliverables:
 - In class demo
 - Printed code and pseudo code
 - Printed graph



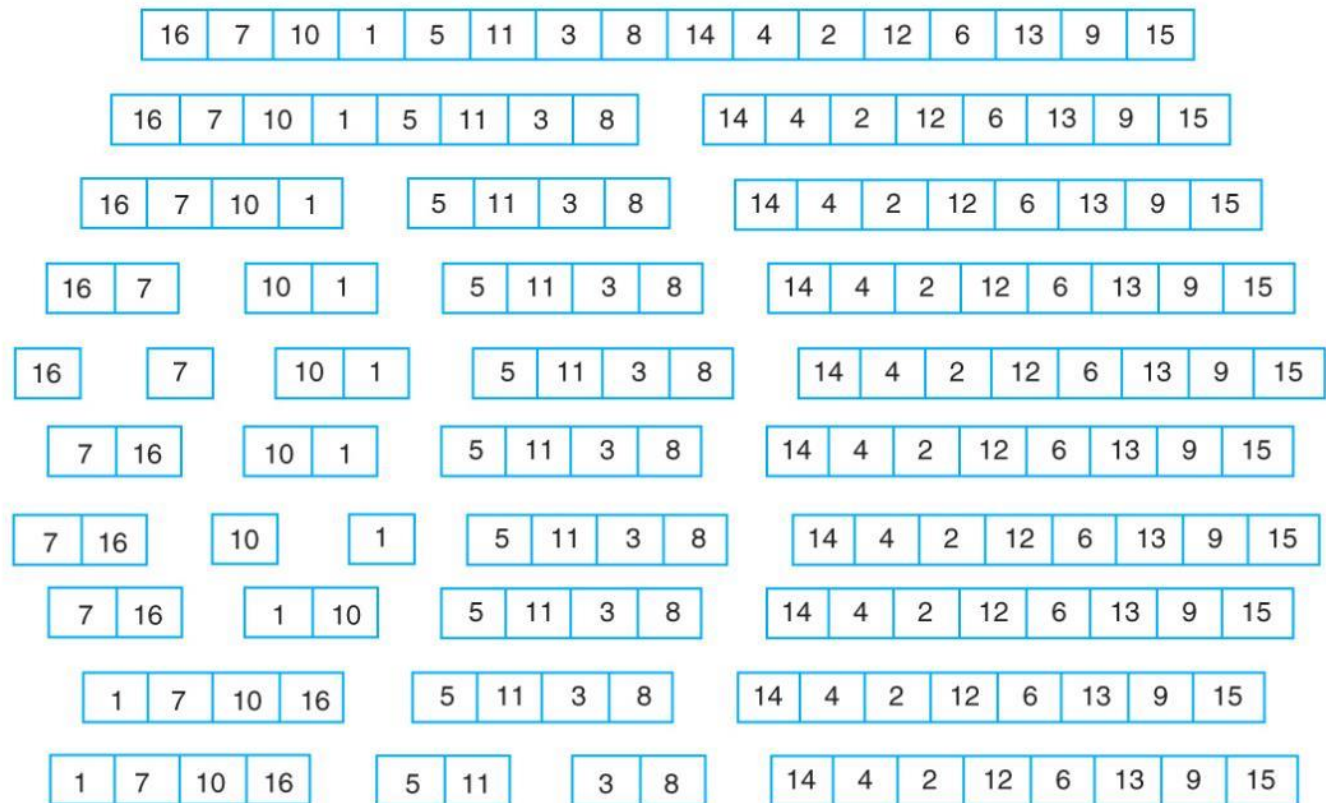
Mergesort

- Old but still in use (40's)
- First divide and conquer sorting algorithm
- Split array $A[0..n-1]$ in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively

Merge Sort Example

■ **FIGURE 4.8**

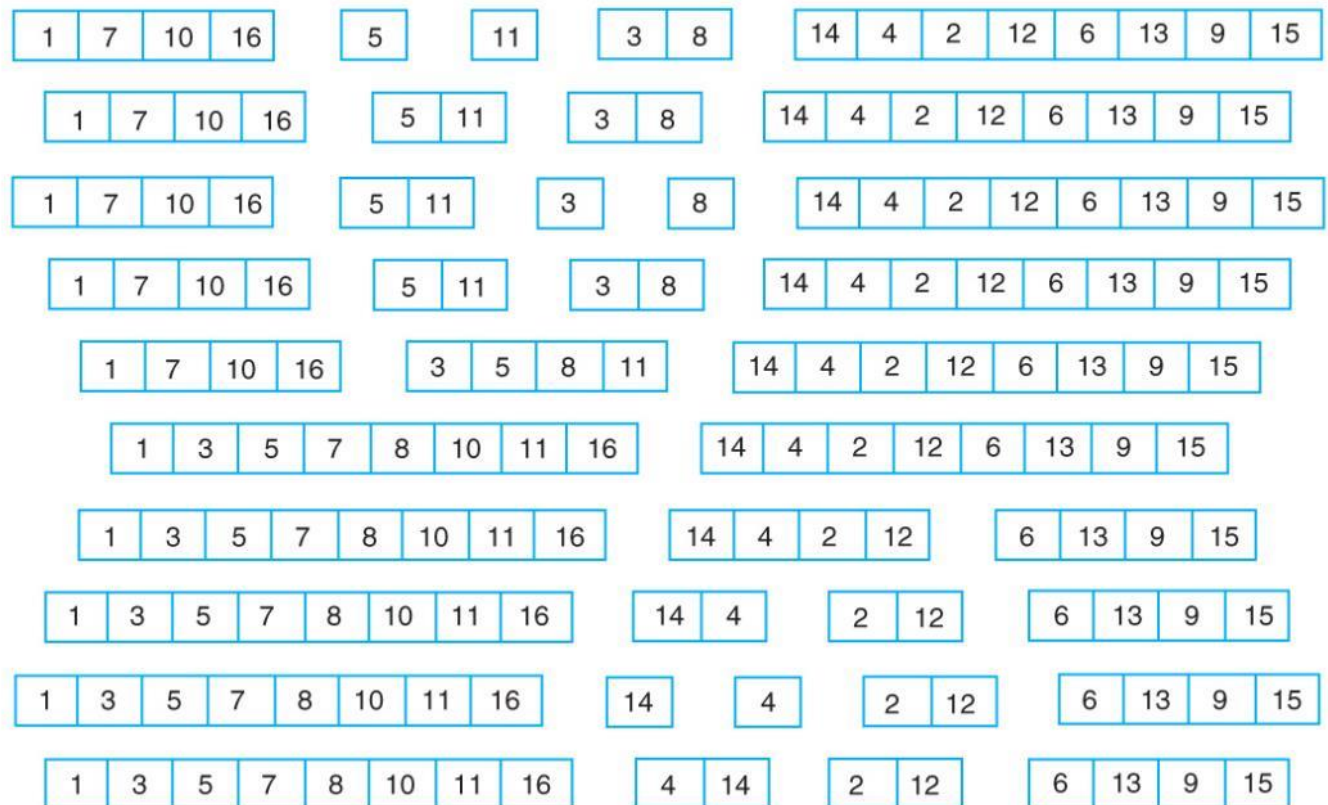
Merge sort
example



Merge Sort Example (continued)

■ **FIGURE 4.8**

Merge sort
example



The Algorithm may look

```
if first < last then
    middle = ( first + last ) / 2
    MergeSort( list, first, middle )
    MergeSort( list, middle + 1, last )
    MergeLists( list, first, middle, middle + 1, last )
end if
```


Pseudocode of Mergesort

ALGORITHM *Mergesort*($A[0..n - 1]$)

//Sorts array $A[0..n - 1]$ by recursive mergesort

//Input: An array $A[0..n - 1]$ of orderable elements

//Output: Array $A[0..n - 1]$ sorted in nondecreasing order

if $n > 1$

 copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$

 copy $A[\lfloor n/2 \rfloor..n - 1]$ to $C[0..\lceil n/2 \rceil - 1]$

Mergesort($B[0..\lfloor n/2 \rfloor - 1]$)

Mergesort($C[0..\lceil n/2 \rceil - 1]$)

Merge(B, C, A)

Pseudocode of Merge

ALGORITHM *Merge*($B[0..p-1]$, $C[0..q-1]$, $A[0..p+q-1]$)

//Merges two sorted arrays into one sorted array

//Input: Arrays $B[0..p-1]$ and $C[0..q-1]$ both sorted

//Output: Sorted array $A[0..p+q-1]$ of the elements of B and C

$i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow 0$

while $i < p$ **and** $j < q$ **do**

if $B[i] \leq C[j]$

$A[k] \leftarrow B[i]$; $i \leftarrow i + 1$

else $A[k] \leftarrow C[j]$; $j \leftarrow j + 1$

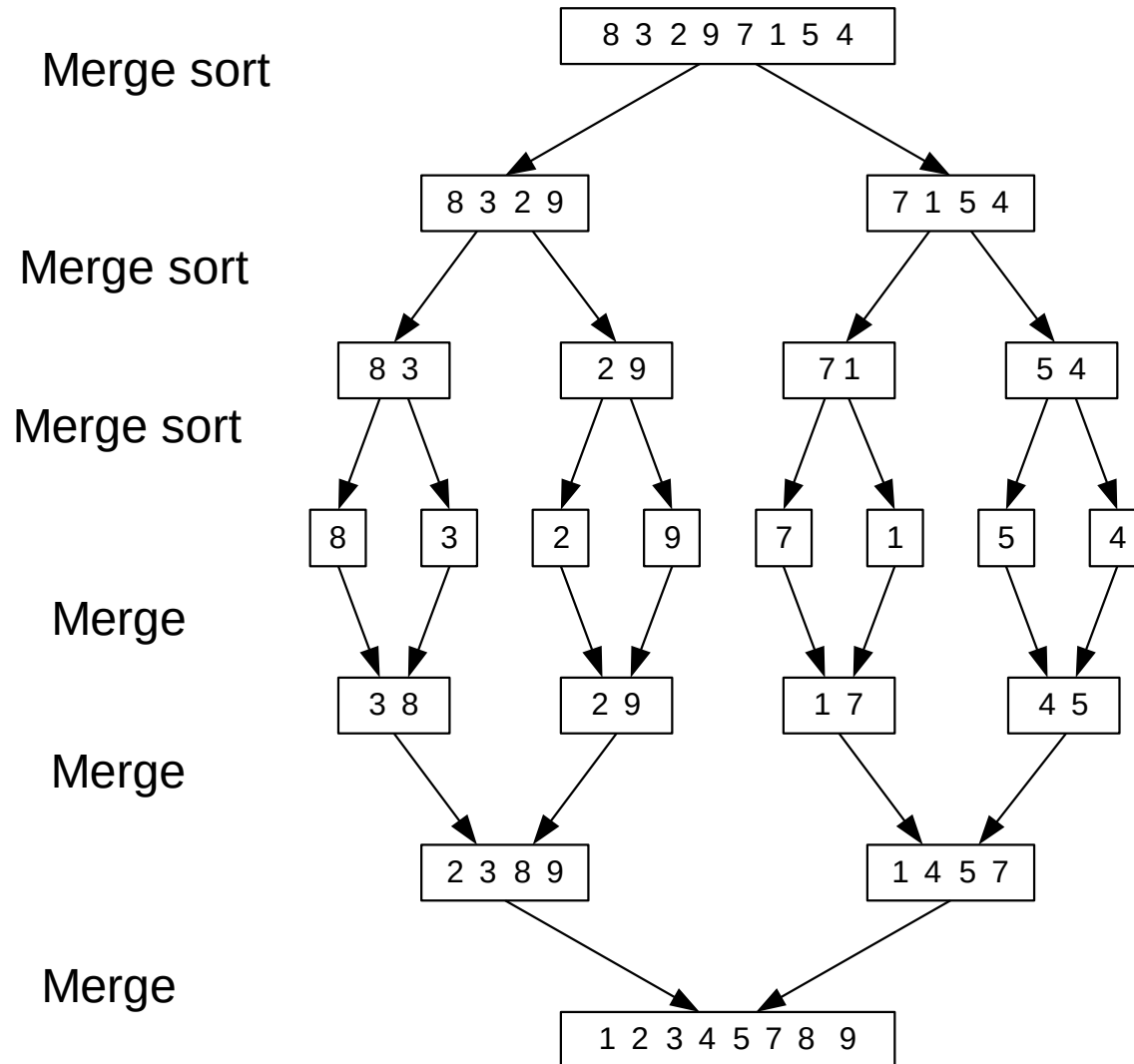
$k \leftarrow k + 1$

if $i = p$

 copy $C[j..q-1]$ to $A[k..p+q-1]$

else copy $B[i..p-1]$ to $A[k..p+q-1]$

Mergesort Example



The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$

MergeSort Analysis

- The worst case is:

$$W(N) = 2W(N/2) + N - 1$$

$$W(0) = W(1) = 0$$

which solves to $W(N) = O(N \lg N)$

- The best case is:

$$B(N) = 2B(N/2) + N/2$$

$$B(0) = B(1) = 0$$

which solves to $B(N) = O(N \lg N)$

Analysis of Mergesort

- All cases have same efficiency: $\Theta(n \log n)$
- Space requirement: $\Theta(N \text{ Log}N)$ (not in-place)
 - Why?

External Sorting

Which $O(n \log n)$ algorithm you use for sorting doesn't matter much until n is so big the data does not fit in memory. Mergesort proves to be the basis for the most efficient *external sorting* programs.

Disks are much slower than main memory, and benefit from algorithms that read and write data in long streams – not random access.

QuickSort

- Definitely a “greatest hit” algorithm
- Prevalent in practice
- Beautiful analysis
- $O(n \log n)$ time “on average”, works in place
 - i.e., minimal extra memory needed

Quicksort

In practice, the fastest *internal* sorting algorithm is Quicksort, which uses *partitioning* as its main idea.

Example: pivot about 10.

Before: 17 12 6 19 23 8 5 10

After:: 6 8 5 10 23 19 12 17

Partitioning places all the elements less than the pivot in the *left* part of the array, and all elements greater than the pivot in the *right* part of the array. The pivot fits in the slot between them.

Note that the pivot element ends up in the correct place in the total order!

Quicksort

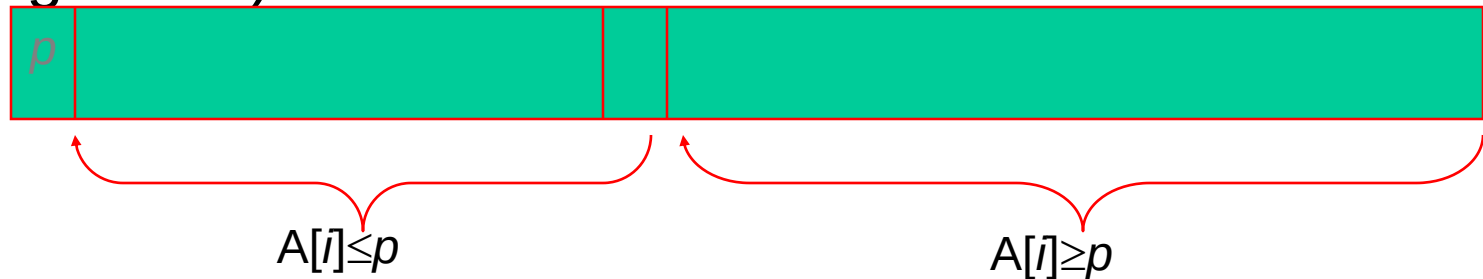
- Quicksort will partition a list into two pieces:
 - Those elements smaller than the pivot value
 - Those elements larger than the pivot value
- Quicksort is then called recursively on both pieces
- No merging phase (combine)



- <https://visualgo.net/bn/sorting>

Quicksort

- Select a *pivot* (partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining $n-s$ positions are larger than or equal to the pivot (see next slide for an algorithm)



- Exchange the pivot with the last element in the first (i.e., $\leq$) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

Why Partition?

Since the partitioning step consists of at most n swaps, takes time linear in the number of keys. But what does it buy us?

1. The pivot element ends up in the position it retains in the final sorted order.
2. After a partitioning, no element flops to the other side of the pivot in the final sorted order.

Thus we can sort the elements to the left of the pivot and the right of the pivot independently, giving us a recursive sorting algorithm!

Quicksort Example

■ **FIGURE 4.9**

Quick sort
example

16	7	10	1	5	11	3	8	14	4	2	12	6	13	9	15	
15	7	10	1	5	11	3	8	14	4	2	12	6	13	9	16	
																P
9	7	10	1	5	11	3	8	14	4	2	12	6	13	15	16	
																P x
6	7	1	5	3	8	4	2	9	10	11	12	14	13	15	16	
																P x x
2	1	5	3	4	6	7	8	9	10	11	12	14	13	15	16	
																P x x
1	2	5	3	4	6	7	8	9	10	11	12	14	13	15	16	
																P x x
1	2	4	3	5	6	7	8	9	10	11	12	14	13	15	16	
x	x															P x x

Quicksort Example (continued)

■ **FIGURE 4.9**
Quick sort
example

1	2	3	4	5	6	7	8	9	10	11	12	14	13	15	16
x	x		P	x	x			x						x	x

1	2	3	4	5	6	7	8	9	10	11	12	14	13	15	16
x	x	x	x	x	x	P		x						x	x

1	2	3	4	5	6	7	8	9	10	11	12	14	13	15	16
x	x	x	x	x	x	x	x	x	P					x	x

1	2	3	4	5	6	7	8	9	10	11	12	14	13	15	16
x	x	x	x	x	x	x	x	x	x	P				x	x

1	2	3	4	5	6	7	8	9	10	11	12	14	13	15	16
x	x	x	x	x	x	x	x	x	x	x	P			x	x

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
x	x	x	x	x	x	x	x	x	x	x	x		P	x	x

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

- 
- Try to develop pseudo code for this algorithm
 - Work in a group of 2 students
 - Test your algorithm on
 - 8 3 2 9 1

Quicksort Pseudocode

Sort(A)

 Quicksort(A, 1, n)

Quicksort(A, low, high)

 if (low < high)

 pivot-location = Partition(A, low, high)

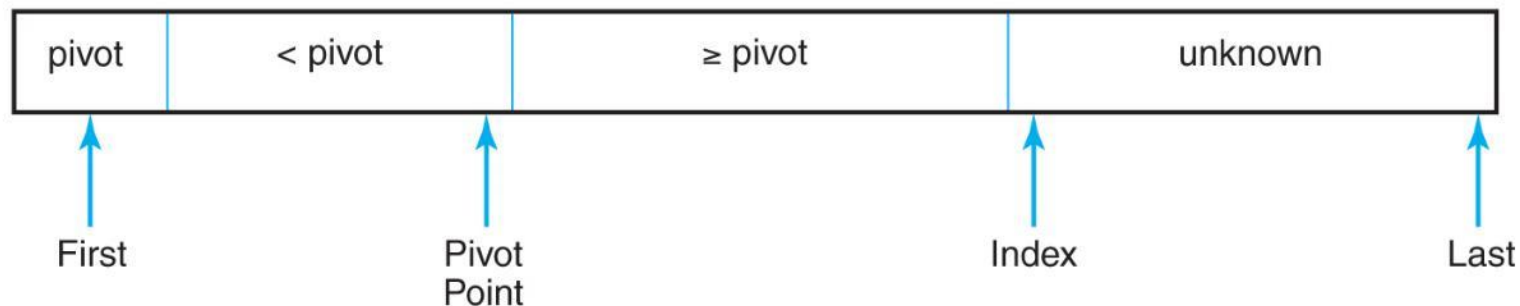
 Quicksort(A, low, pivot-location - 1)

 Quicksort(A, pivot-location+1, high)

Partitioning Process

- The algorithm moves through the list comparing values to the pivot
- During this process, there are sections of elements as indicated below

■ **FIGURE 4.10**
Relationship
between the
indices and
element values in
`PivotList`



Partition Implementation

```
Partition(A,low,high)
    pivot = A[low]
    leftwall = low
    for  $i = \text{low}+1$  to high
        if ( $A[i] < \text{pivot}$ ) then
            leftwall = leftwall+1
            swap( $A[i], A[\text{leftwall}]$ )
    swap( $A[\text{low}], A[\text{leftwall}]$ )
```



Quicksort Example

Trace

5 3 1 9 8 2 4 7

Modify the code to sort descending order



The Importance of the Pivot

Q : running time of QuickSort ?

A : depends on the quality of the pivot.

A decorative header with a blue abstract background featuring wavy, liquid-like patterns.

Suppose we run QuickSort on some input, and, magically, every recursive call chooses the median element of its subarray as its pivot. What's the running time in this case?

Suppose we run QuickSort on some input, and, magically, every recursive call chooses the median element of its subarray as its pivot. What's the running time in this case?

☐ Not enough information to answer question

☐ $\theta(n)$

☒ $\theta(n \log n)$

☐ $\theta(n^2)$

Reason : Let $T(n)$ = running time on arrays of size n .

Because pivot = median
choosePivot
partition

Then : $T(n) \leq 2T(n/2) + \theta(n)$
 $\Rightarrow T(n) = \theta(n \log n)$ [like MergeSort]

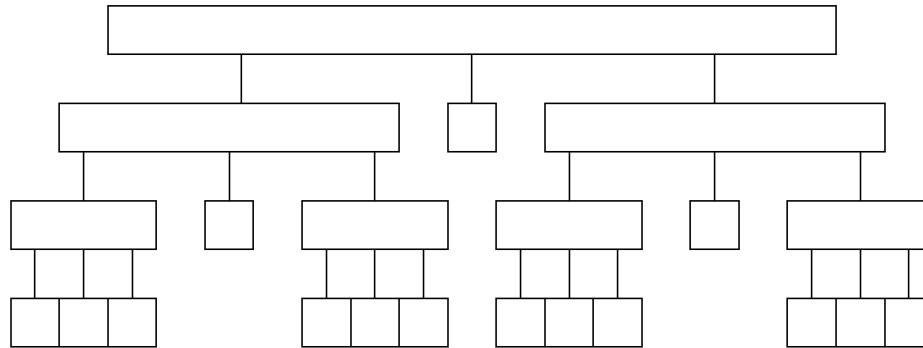
Best Case for Quicksort

Since each element ultimately ends up in the correct position, the algorithm correctly sorts. But how long does it take?

The best case for *divide-and-conquer* algorithms comes when we split the input as evenly as possible. Thus in the best case, each subproblem is of size $n/2$.

The partition step on each subproblem is linear in its size. Thus the total effort in partitioning the 2^k problems of size $n/2^k$ is $O(n)$.

Best Case Recursion Tree



The total partitioning on each level is $O(n)$, and it takes $\lg n$ levels of perfect partitions to get to single element subproblems. When we are down to single elements, the problems are sorted. Thus the total time in the best case is $O(n \lg n)$.



Analysis of Quicksort

- Using master theorem?
- Best case: split in the middle

The Master Method

If $T(n) \leq aT\left(\frac{n}{b}\right) + O(n^d)$

then

$$T(n) = \begin{cases} O(n^d \log n) & \text{if } a = b^d \quad (\text{Case 1}) \\ O(n^d) & \text{if } a < b^d \quad (\text{Case 2}) \\ O(n^{\log_b a}) & \text{if } a > b^d \quad (\text{Case 3}) \end{cases}$$

QuickSort Analysis

- The best case is:

$$B(N) = 2B(N/2) + O(N)$$

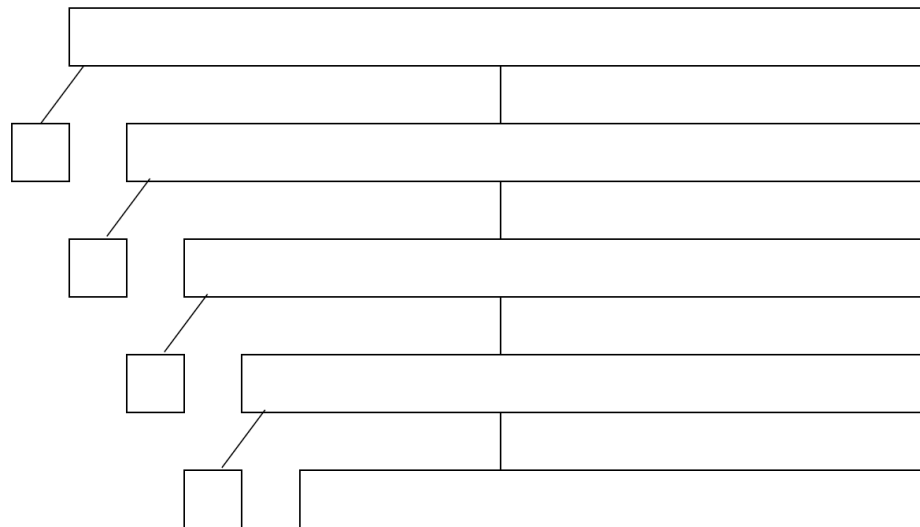
which solves to $B(N) = O(N \lg N)$

Analysis of Quicksort

- Using master theorem
- Best case: split in the middle — $\Theta(n \log n)$

Worst Case for Quicksort

Suppose instead our pivot element splits the array as unequally as possible. Thus instead of $n/2$ elements in the smaller half, we get zero, meaning that the pivot element is the biggest or smallest element in the array.



QuickSort Analysis

- The worst case is:
$$W(N) = W(N-1) + C$$
- which solves to $W(N) = O(N^2)$



Now we have $n-1$ levels, instead of $\lg n$, for a worst case time of $\Theta(n^2)$, since the first $n/2$ levels each have $\geq n/2$ elements to partition.

To justify its name, Quicksort had better be good in the average case. Showing this requires some intricate analysis. The divide and conquer principle applies to real life. If you break a job into pieces, make the pieces of equal size!

Pick a Better Pivot

Having the worst case occur when they are sorted or almost sorted is *very bad*, since that is likely to be the case in certain applications.

To eliminate this problem, pick a better pivot:

1. Use the middle element of the subarray as pivot.
2. Use a *random* element of the array as the pivot.
3. Perhaps best of all, take the median of three elements (first, last, middle) as the pivot. Why should we use median instead of the mean?

Whichever of these three rules we use, the worst case remains $O(n^2)$.

Randomized Guarantees

Randomization is a very important and useful idea. By either picking a random pivot or scrambling the permutation before sorting it, we can say:

“With high probability, randomized quicksort runs in $\Theta(n \lg n)$ time.”

Where before, all we could say is:

“If you give me random input data, quicksort runs in expected $\Theta(n \lg n)$ time.”

Random Pivots

Key Question : how to choose pivots ? BIG IDEA : RANDOM PIVOTS!

That is : in every recursive call, choose the pivot randomly.
(each element equally likely)

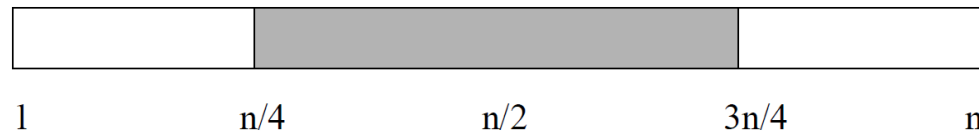
Hope : a random pivot is “pretty good” “often enough”.

Intuition : 1.) if always get a 25-75 split, good enough for $O(n \log(n))$

2.) half of elements give a 25-75 split or better

Intuition: The Average Case for Quicksort

Suppose we pick the pivot element at random in an array of n keys.



Half the time, the pivot element will be from the center half of the sorted array.

Whenever the pivot element is from positions $n/4$ to $3n/4$, the larger remaining subarray contains at most $3n/4$ elements.

Analysis of Quicksort

- Best case: split in the middle — $\Theta(n \log n)$
- Worst case: sorted array! — $\Theta(n^2)$
- Average case: random arrays — $\Theta(n \log n)$
- Improvements:
 - better pivot selection: median of three partitioning
 - switch to insertion sort on small sub files
 - elimination of recursionThese combine to 20-25% improvement
- Considered the method of choice for internal sorting of large files ($n \geq 10000$)

Stable Sort

- A property of sorts
- If a sort guarantees the relative order of equal items stays the same then it is a *stable sort*
- $[7_1, 6, 7_2, 5, 1, 2, 7_3, -5]$
 - subscripts added for clarity
- $[-5, 1, 2, 5, 6, 7_1, 7_2, 7_3]$
 - result of stable sort

HW 5

Due November 11th

- Compare one of the studied sorted algorithm (Bubble, insertion, Merge, Quick) with one of the followings:(Radix , Shell, heap, bucket). (in terms of complexity, needed memory)
- Find the complexity of the new one.
- Implement a program to compare complexity of each and graph the results (execution time of each as a function of input size (10K, 100K, 1 M)
- Deliverables:
 - In class demo
 - One page comparison
 - Graph
 - Sample runs

Jeffrey J. McConnell

Analysis *of* Algorithms

Second Edition
An Active Learning Approach



Chapter 5

Numeric Algorithms



Chapter Outline

- Calculating polynomials
- Matrix multiplication
- Linear equations



Prerequisites

- Before beginning this chapter, you should be able to:
 - Do simple algebra
 - Evaluate polynomials
 - Describe growth rates and order



Goals

- At the end of this chapter you should be able to:
 - Evaluate a polynomial by Horner's method
 - Explain matrix multiplication
 - Explain the Gauss-Jordan method for solving systems of linear equations

Numeric Algorithm Efficiency

- Numeric programs will do a particular calculation a large number of times
- Small improvements can result in significant time savings because of how many times the calculation is done

Numeric Analysis Basics

- We count additions and multiplications
- Because additions are typically much faster than multiplications, reducing the multiplications by increasing the additions can still result in an improvement
- The analysis will be based on the highest power in a polynomial or the size of the matrices we are working with

Calculating Polynomials

- We will use a generic polynomial form of:

$$p(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_0$$

where the coefficient values are known constants

- The value of x will be the input and the result is the value of the polynomial using this x value

- 
- A decorative header with a blue abstract pattern of flowing, wavy lines.
- Develop algorithm
 - Work in a group of two

Standard Evaluation Algorithm

```
result = a[0] + a[1]*x
xPower = x
for i = 2 to n do
    xPower = xPower * x
    result = result + a[i]*xPower
end for
return result
```



Do the Analysis

how many operations we have??

- Before the loop, there is
- The `for` loop
- The Total

Analysis

- Before the loop, there is
 - One multiplication
 - One addition
- The `for` loop is done $N-1$ times
 - There are two multiplications in the loop
 - There is one addition in the loop
- There are a total of
 - $2N - 1$ multiplications
 - N additions

Horner's Method

- Based on the factorization of a polynomial
- The generic equation is factored as:

$$p(x) = (\{ \cdots [(a_n x + a_{n-1}) * x + a_{n-2}] * x + \cdots + a_2 \} * x + a_1) * x + a_0$$

- For example, the equation:

$$p(x) = x^3 - 5x^2 + 7x - 4$$

- would be factored as:

$$p(x) = [(x - 5) * x + 7] * x - 4$$

- 
- Develop algorithm
 - Work in a group of two

Horner's Method Algorithm

```
result = a[n]
for i = n - 1 down to 0 do
    result = result * x
    result = result + a[i]
end for
return result
```



Do the Analysis

- The `for` loop??
- The Total?

Analysis

- The `for` loop is done N times
 - There is one multiplication in the loop
 - There is one addition in the loop
- There are a total of
 - N multiplications
 - N additions
- Saves $N - 1$ multiplications over the standard algorithm

Polynomial Algorithm Comparison

- For the example polynomial:
 - Standard algorithm:
 - 5 multiplications and 3 additions
 - Horner's method
 - 3 multiplications and 3 additions

$$p(x) = x^3 - 5x^2 + 7x - 4$$

Polynomial Algorithm Comparison

- In general, for a polynomial of degree N :
 - Standard algorithm:
 - $2N - 1$ multiplications and N additions
 - Horner's method
 - N multiplications and N additions